

Supply-Chain Security Scanner

A Unified CLI for Multi-Provider
Package Vulnerability Analysis

Abhinav Thakur

Msc. Software Engineering

Academic Year 2025–2026

Declaration

I declare that this project and the work presented in it is my own. Where I have drawn on the work of others I have acknowledged this with appropriate references. This project has not been submitted for assessment at any other institution. Grammarly was used only for grammar and spelling correction; all technical content, analysis, and conclusions are my own.

Signature: Abhinav Thakur

Date: 13 May 2026

Abstract

Package managers are tools that automate the download, installation, and management of software libraries and their dependencies within a development project. Widely used open-source package managers include `npm` (Node Package Manager) for JavaScript, `pip3` for Python, and `yarn` and `pnpm` as `npm` alternatives; proprietary alternatives also exist but the dominant tools in use today are open-source.

Software supply-chain attacks increased sharply after 2020. Packages on PyPI and `npm` are installed billions of times each week, yet most developers install them through a single command with no automatic safety check. This project builds `unified`, a command-line tool that wraps four package managers (`npm`, `pip3`, `yarn`, `pnpm`) and queries four independent security databases before allowing any installation or upgrade to proceed. The databases are OSV.dev, the GitHub Advisory Database, Sonatype OSS Index, and VirusTotal.

The tool resolves the exact package version before downloading the artifact hash, closing a time-of-check to time-of-use window that exists in naive implementations. It scans transitive dependencies introduced by an install and rolls back the entire operation if any new dependency is blocked. A SQLite database tracks every package managed through the tool. A JSON TTL cache avoids redundant provider requests during repeated operations.

The test suite covers 114 cases across unit, integration, and regression tests. All four architectural security properties are verified by dedicated regression tests. The tool is designed to run on Windows, macOS, and Linux; development and all testing were conducted on Windows 11, with cross-platform portability ensured through `shutil.which` rather than hardcoded paths.

Acknowledgements

I thank my supervisor and the open-source communities behind OSV.dev, the GitHub Advisory Database, Sonatype OSS Index, and VirusTotal for providing free public APIs that made this project feasible.

Contents

Declaration	i
Abstract	ii
Acknowledgements	iii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Research Aims and Objectives	2
1.3.1 Aims	2
1.3.2 Research Questions	2
1.3.3 Objectives	3
1.4 Scope and Limitations	3
2 Research Methodology	4
2.1 Introduction	4
2.2 Development Approach	4
2.3 Toolchain and Environment	5
3 Literature Review	6
3.1 Introduction	6
3.2 Supply-Chain Attack Taxonomy	6
3.3 Existing Defences and Their Gaps	6
3.4 Vulnerability Databases	7
3.5 Related Work	8
4 System Design	9
4.1 Introduction	9
4.2 Requirements	10
4.2.1 Functional Requirements	10
4.2.2 Non-Functional Requirements	10

4.3	Architecture	11
4.4	Component Design	12
4.4.1	Manager Abstraction	12
4.4.2	SecurityAggregator	12
4.4.3	SecurityScanCache	12
4.4.4	PackageCacheDB	13
4.4.5	Environment Variables	13
4.5	Implementation	13
4.5.1	TOCTOU Fix	13
4.5.2	Transitive Dependency Scanning	14
4.5.3	OSV CVSS Parser	14
4.5.4	Fail-Closed Exception Handling	15
5	Testing and Evaluation	16
5.1	Introduction	16
5.2	Testing	16
5.2.1	Test Strategy	16
5.2.2	Test Suite Summary	17
5.2.3	Regression Tests for Security Properties	17
5.3	Evaluation	18
6	Conclusions	21
6.1	Introduction	21
6.2	Reflection	21
6.3	Recommendations	22
	Glossary	25
	List of Abbreviations	26
A	CLI Command Reference	27
A.1	Setup	27
A.2	Commands	27
A.3	Environment Variable Quick Reference	28

List of Tables

3.1 Comparison of Public Vulnerability Databases 7

4.1 Functional Requirements 10

4.2 Non-Functional Requirements 10

4.3 Security Decision Thresholds 11

4.4 PackageCacheDB Schema 13

4.5 Environment Variables 13

5.1 Test Suite Summary by Module 17

5.2 Manual Evaluation—Known Vulnerable Packages 19

List of Figures

4.1	High-level architecture — data flow for <code>install</code> and <code>upgrade</code>	11
4.2	Manager class hierarchy	12
4.3	Transitive dependency scan and rollback sequence	14

Chapter 1

Introduction

1.1 Background and Motivation

In December 2020 the SolarWinds Orion update mechanism delivered malicious code to roughly 18,000 organisations [1]. Two years earlier, the `event-stream` npm package was quietly transferred to a new maintainer who inserted a payload targeting one specific Bitcoin wallet [2]. Maintainers are external companies or individuals who own or manage packages on public registries; they are trusted implicitly by the package ecosystem, but that trust is not always warranted. Both attacks exploited the same structural weakness: downstream consumers trusted upstream packages without verifying them independently.

Package managers are tools that automate the installation, upgrade, and removal of software libraries from centralised registries. Common open-source package managers include `npm` and `Yarn` (JavaScript), `pip` (Python), `Cargo` (Rust), and `RubyGems` (Ruby); proprietary or enterprise-controlled equivalents include JFrog Artifactory and GitHub Packages.

The scale of the risk has grown as package registries have grown. As of 2024 the npm registry hosts over 2.5 million packages and records roughly 10 billion downloads per week [3]. The Python Package Index (PyPI) serves comparable scale, with hundreds of billions of downloads annually [12]. A single malicious package that achieves wide adoption can compromise an enormous number of systems before detection.

More recent incidents show that the threat is still active. On 31 March 2026, attackers compromised an `axios` maintainer account on npm and published malicious releases that introduced a hidden dependency, `plain-crypto-js`, which delivered a cross-platform remote access trojan during installation. Because `axios` is so widely used, the incident showed how quickly trust in a single package can become a wider ecosystem problem [13]. In April 2026, attackers similarly compromised SAP-owned npm packages in a targeted supply-chain attack [14]. That same month, the Bitwarden

CLI npm package was tampered with to harvest developer credentials including npm tokens, SSH keys, and cloud platform secrets, illustrating that even security-focused tools are not immune [15].

Existing package managers offer little protection at install time. Running `npm install <package>` or `pip install <package>` will fetch and execute arbitrary code with no prior safety check against any vulnerability database. Some organisations address this through separate audit steps (`npm audit`, `pip-audit`), but these run after installation, by which point code may have already executed build hooks. This project takes a different position: check first, install only if the check passes.

1.2 Problem Statement

The core problem is that developers have no built-in way to verify package safety against multiple independent data sources before installation. Existing tools either require a separate manual step, check only one source, or do not integrate into the install workflow itself. Naive scanner implementations also contain a time-of-check to time-of-use (TOCTOU) race: the version scanned may differ from the version installed if the registry updates between the two operations.

Transitive dependencies make the problem worse. Installing `django` also installs `sqlparse` and `asgiref`. A user who consents to `django` has not explicitly consented to those dependencies, yet a malicious transitive package poses equal risk. No mainstream package manager rollback mechanism targets transitive security failures automatically.

1.3 Research Aims and Objectives

1.3.1 Aims

The aim of this project is to design, implement, and evaluate a CLI tool that integrates multi-provider security scanning directly into the package install and upgrade workflow, with automatic rollback for transitive violations.

1.3.2 Research Questions

The following research questions guide the design and evaluation of this project:

1. *What* security weaknesses exist in the install workflows of mainstream open-source package managers that allow malicious or vulnerable packages to be installed without prior verification?

2. *How* can multi-provider vulnerability scanning be integrated into a package installation workflow without introducing a time-of-check to time-of-use (TOCTOU) race condition?
3. *To what extent* does pre-install multi-provider security scanning reduce the risk of malicious or vulnerable package installation compared with post-install auditing approaches?

1.3.3 Objectives

The specific objectives of this project are:

- Scan packages against multiple independent vulnerability databases before installation.
- Eliminate the time-of-check to time-of-use (TOCTOU) race between security scanning and package installation.
- Detect and block malicious transitive dependencies automatically.
- Maintain a consistent record of all packages managed through the tool.
- Cache scan results to avoid redundant network calls during repeated operations.
- Support the npm, pip3, Yarn, and pnpm package manager ecosystems.

1.4 Scope and Limitations

The tool covers Python (PyPI) and JavaScript (npm, yarn, pnpm) ecosystems. It does not cover RubyGems, Cargo, or Go modules. VirusTotal scanning requires an API key; when the key is absent and scans with the remaining three providers instead.

The tool scans before install. It does not continuously monitor already-installed packages for newly published advisories. That is a separate and important problem, but it falls outside the scope of this project.

Chapter 2

Research Methodology

2.1 Introduction

This chapter describes how the project was conducted: the development process, the tools used, and the decisions that shaped the implementation.

This research uses a quantitative, experimental methodology. The work involves designing, implementing, and evaluating a concrete software artefact—the **unified** CLI tool—against defined security criteria. The measurable outcomes are test pass rates, detection accuracy across vulnerability providers, and timing data for install operations. An experimental approach fits because the project produces a functional tool whose security properties can be directly and objectively measured.

2.2 Development Approach

The project used an iterative development approach. An initial prototype wrapped only `pip3` and queried a single provider (VirusTotal). Each subsequent iteration added a capability: a second provider, a second manager, the TOCTOU fix, transitive scanning, and so on. This made it possible to evaluate each feature in isolation before integrating it.

Test-driven development was applied specifically for the four architectural security properties captured in `test_security_fixes.py`: the TOCTOU fix, transitive scanning with rollback, the consistent package database, and fail-closed exception handling. The regression tests for these properties were written before the corresponding implementation. Other modules were developed test-after—tests were added once the behaviour was understood rather than before. This distinction is reflected in Section 6.2, where the OSV CVSS parser bug is noted as one that was not caught until a test was written against a real provider response. The test-first approach was chosen for the architectural properties because security invariants are easy to get subtly wrong and

hard to verify by inspection alone.

The development environment was Windows 11. Cross-platform compatibility was ensured by using `shutil.which` for executable resolution rather than hardcoded paths, and by running the test suite on each push. The tool was not independently verified on macOS or Linux; cross-platform correctness rests on the portability of `shutil.which` and the Python standard library rather than direct multi-OS testing.

2.3 Toolchain and Environment

The implementation language is Python 3.12. Key dependencies are: `requests` for HTTP, `colorama` for cross-platform terminal colour, `tabulate` for table formatting, `python-dotenv` for environment variable loading, and `pytest` for testing. The project uses `setuptools` with a `pyproject.toml` for packaging.

Security provider access is purely via their public HTTP APIs. No proprietary SDK is used, which keeps the dependency footprint small and the network behaviour testable through standard HTTP mocking.

Chapter 3

Literature Review

3.1 Introduction

This chapter surveys supply-chain attack techniques, the databases that track them, and the tools that already exist to defend against them. The goal is to establish what already exists in this space and explain where this project's design departs from it.

3.2 Supply-Chain Attack Taxonomy

Ladisa et al. produced a taxonomy of software supply-chain attacks that identifies eight primary attack vectors [4]. The most relevant to this project are: injecting malicious code into a published package (typosquatting or account compromise), and dependency confusion attacks that exploit name resolution order to serve a private-namespace package from the public registry.

The 2021 Log4Shell vulnerability (CVE-2021-44228) demonstrated a different failure mode: a legitimate, widely-used library contained a critical flaw that had existed undetected for years [5]. This is not a supply-chain attack in the narrow sense, but it shows why scanning against known-vulnerability databases matters even for packages that are not themselves malicious.

The event-stream incident of 2018 is perhaps the clearest illustration of account-takeover risk. The original author transferred the package to a stranger who published a version containing a targeted payload, which was then downloaded by thousands of projects before detection [2].

3.3 Existing Defences and Their Gaps

`npm audit` and `pip-audit` are the most widely used first-party scanners. Both query a single vulnerability database and both run after installation. They are useful for

auditing an existing dependency tree, but they do not block the install itself, and they do not cross-check against VirusTotal or OSS Index.

Snyk and Socket.dev are commercial products that provide pre-install scanning with broader coverage. They address many of the gaps in the first-party tools, but they require accounts and their free tiers impose rate limits that make them impractical for CI environments without subscription. This project targets the zero-cost case: meaningful multi-provider scanning with no mandatory account.

None of the above tools address the TOCTOU problem explicitly in their public documentation. The assumption appears to be that scanning and installation happen close enough in time that the registry will not change between the two. This holds in practice most of the time, but not during active attacks where an adversary pushes a malicious version immediately after the scan completes.

3.4 Vulnerability Databases

Table 3.1 compares the four databases used by this project.

Table 3.1: Comparison of Public Vulnerability Databases

Database	Coverage	API Type	Auth	Rate Limit (free)
OSV.dev	PyPI, npm, Go, Rust, others	REST POST	No	No documented limit
GitHub Advisory	npm, pip, Composer, Rust, others	REST GET	Optional	60 req/h unauth
OSS Index	PyPI, npm (PURL)	REST POST	Optional	~128 req/h
VirusTotal	File hashes (SHA-256)	REST GET	Yes (free key)	4 lookups/min

OSV.dev is run by Google and covers the broadest set of ecosystems [6]. It returns structured vulnerability records with CVSS severity information, though the format of the severity field is inconsistent: some records embed CVSS vector strings, others embed numeric scores directly. Handling both formats correctly required specific implementation work, described in Section 4.5.3.

VirusTotal is the only database in this project that operates on artifact hashes rather than package metadata. It can in principle detect a package whose published metadata appears clean but whose downloaded file has a hash already known to VirusTotal’s threat-intelligence network—an attack vector that purely metadata-based advisory databases would not cover. Detection depends on the hash being present in VirusTotal’s database, so novel or targeted malware may still evade it.

3.5 Related Work

Zimmermann et al. analysed the npm dependency graph and found that a single popular package is reachable from 40% of all npm packages through the transitive dependency tree [7]. This quantifies the blast radius of a single compromised package and directly motivates the transitive scanning feature implemented in this project.

Duan et al. studied malicious packages in PyPI and npm from 2017 to 2022 and found that the majority used one of four techniques: typosquatting, dependency confusion, account takeover, or direct injection via compromised build pipelines [8]. Their taxonomy informed the design decision to check both package metadata (via advisory databases) and the actual artifact hash (via VirusTotal).

Chapter 4

System Design

4.1 Introduction

This chapter covers the requirements, architecture, component design, and notable implementation details of the supply-chain security scanner.

4.2 Requirements

4.2.1 Functional Requirements

Table 4.1: Functional Requirements

ID	Requirement
FR1	Install, upgrade, uninstall, search, and list packages across npm, pip3, yarn, pnpm.
FR2	Before install/upgrade, query OSV.dev, GitHub Advisory, OSS Index, and VirusTotal.
FR3	Block installation if any provider reports a critical or malicious finding.
FR4	Warn and prompt the user if any provider reports a high or medium finding.
FR5	Allow installation only when no findings are detected and at least two providers responded.
FR6	Resolve the exact package version before downloading the artifact (TOCTOU fix).
FR7	Scan transitive dependencies introduced by an install; roll back if any are blocked.
FR8	Maintain a SQLite database of packages installed through the tool.
FR9	Cache scan results with a configurable TTL to avoid redundant API calls.
FR10	Support <code>-force</code> to override a block and <code>-no-security</code> to skip scanning.
FR11	Support <code>-dry-run</code> for <code>upgrade -all</code> as a CI gate.
FR12	Write JSON and Markdown security reports to <code>security_reports/</code> .

4.2.2 Non-Functional Requirements

Table 4.2: Non-Functional Requirements

ID	Requirement
NFR1	The tool must run on Windows, macOS, and Linux.
NFR2	A scan exception must block the action (fail-closed), not silently allow it.
NFR3	Scan results must be reproducible given the same package, version, and provider auth state.
NFR4	HTTP requests to providers must time out within 12 seconds with automatic retry.
NFR5	The test suite must run without any live network connections.

4.3 Architecture

Figure 4.1 shows the high-level data flow for the `install` command. The same pipeline applies to `upgrade`.

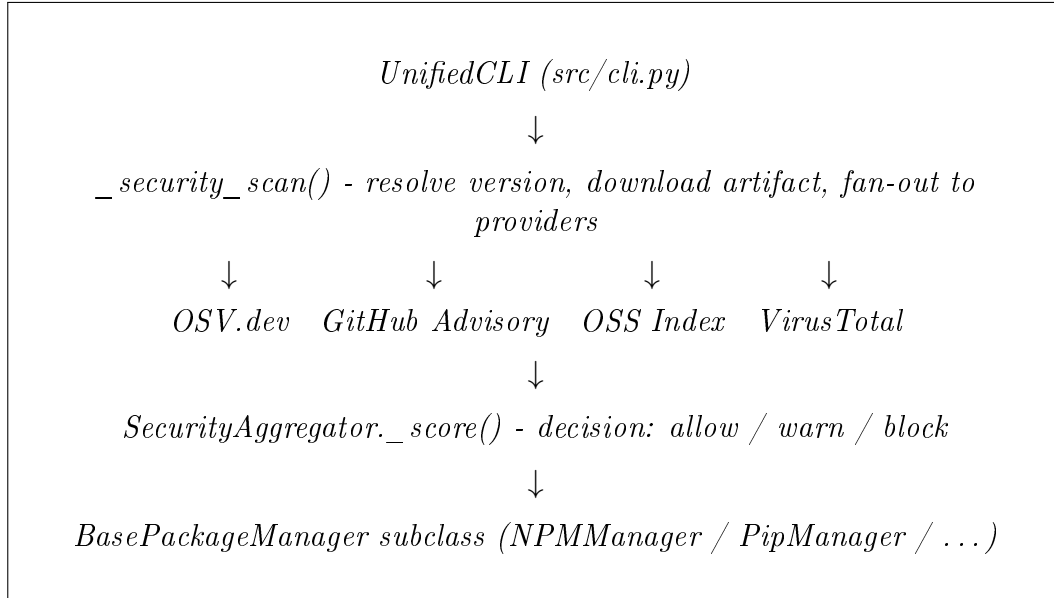


Figure 4.1: High-level architecture — data flow for `install` and `upgrade`

The entry point is `unified` (installed via `pip`), which calls `UnifiedCLI` in `src/cli.py`. The CLI layer parses arguments, resolves the target manager, and dispatches to handler methods. The security pipeline runs as a synchronous pre-check before the underlying package manager is invoked.

The decision thresholds are shown in Table 4.3.

Table 4.3: Security Decision Thresholds

Condition	Decision	Action
Any vt-malicious finding, or any critical-severity finding	block	Installation aborted.
Any high or medium severity finding	warn	User prompted; proceeds if confirmed (or <code>-force</code>).
Any low-severity finding only	warn	Presence of any finding prevents a clean allow.
Unknown-severity finding only (CVSS vector unresolvable)	warn	Treated conservatively; user prompted.
No findings AND coverage ≥ 2 providers	allow	Installation proceeds.
No findings AND coverage < 2 providers	warn	Insufficient confidence; user prompted.
<code>SecurityAggregator.analyze()</code> block raises any exception		Fail-closed; installation aborted.

4.4 Component Design

4.4.1 Manager Abstraction

All four package managers inherit from `BasePackageManager`, an abstract base class that defines six methods: `list_packages`, `install_package`, `search_package`, `upgrade_package`, `check_outdated`, and `uninstall_package`. Figure 4.2 shows the class hierarchy.

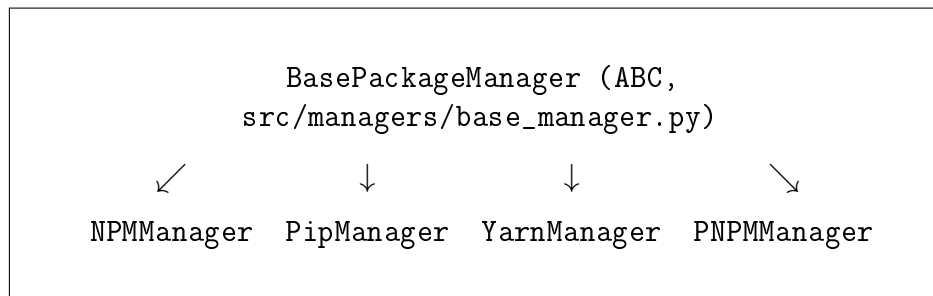


Figure 4.2: Manager class hierarchy

The base class provides `_run_command`, which handles Windows path resolution through `shutil.which`, and `get_latest_registry_version`, which defaults to querying the npm registry. `PipManager` overrides the latter to query PyPI instead. The three npm-family managers share `_search_npm_registry` as a static helper.

4.4.2 SecurityAggregator

`SecurityAggregator` in `src/utils/security_aggregator.py` calls all four providers and aggregates their results into a single decision. The cache key includes the schema version, manager, package name, version, file hash, and a fingerprint of which API credentials are configured. This last element prevents a cache entry from an unauthenticated run being returned in an authenticated context, which could mask findings that only appear when the provider grants full access.

4.4.3 SecurityScanCache

`SecurityScanCache` in `src/utils/security_cache.py` is a JSON file stored at `.security_scan_cache` in the working directory. Each entry stores the result and an `expires_at` Unix timestamp. The default TTL is 600 seconds (10 minutes), configurable via `SECURITY_CACHE_TTL_SECONDS`. On read, expired entries are deleted before returning `None`, so stale data does not accumulate.

4.4.4 PackageCacheDB

PackageCacheDB in `src/utils/package_cache.py` is a SQLite database with a single table. Table 4.4 shows the schema.

Table 4.4: PackageCacheDB Schema

Column	Type	Notes
id	INTEGER PK AUTOINCREMENT	Internal row identifier
name	TEXT NOT NULL	Package name
version	TEXT	Installed version
manager	TEXT	npm, pip3, yarn, or pnpm
(name, version, manager)	UNIQUE	Prevents duplicate records

The `update_package_version` method performs a delete-then-insert rather than an `UPDATE` to avoid `UNIQUE` constraint violations when the same package exists under multiple version records. This upsert pattern also handles the case where the package was never installed through `unified`; the insert adds the row from scratch.

4.4.5 Environment Variables

Table 4.5: Environment Variables

Variable	Purpose	Required
VIRUSTOTAL_API_KEY	VirusTotal file-hash scanning	No
OSSINDEX_USERNAME / OSSINDEX_TOKEN	OSS Index auth	No
GITHUB_TOKEN	Raises GitHub Advisory rate limit	No
SECURITY_CACHE_TTL_SECONDS	Override default 600 s TTL	No

4.5 Implementation

This section documents the four implementation decisions that had the most impact on security correctness.

4.5.1 TOCTOU Fix

The time-of-check to time-of-use problem arises when a scanner resolves version A during the check phase, but the package manager installs version B because the registry updated in the interval. The fix: resolve the exact version first, pass it to the hash download, and pass the same version-pinned specifier to the package manager (`requests==2.31.0` for pip, `lodash@4.17.21` for npm).

`_resolve_scan_version()` checks whether the user embedded a version in the package name, falls back to installed-version lookup, then queries the registry. The

resolved version is stored in `_last_resolved_version` before any provider call. The version extraction regex `r'(?<=.)[@]=?(\\d[^\\s]*)$'` correctly handles scoped packages like `@babel/core`—the leading `@` is only a separator when preceded by at least one character.

4.5.2 Transitive Dependency Scanning

The approach is a state diff. `_capture_package_snapshot()` calls `list_packages()` on the target manager and stores a dict of lowercase name to version. After the install succeeds, the snapshot is taken again. Any package in the after-snapshot but not in the before-snapshot is a newly introduced dependency.

Each new package (excluding the top-level one the user requested) is scanned through `SecurityAggregator.analyze()`. If any returns `"block"`, `_scan_and_rollback_transitive()` calls `uninstall_package()` on every new package including the top-level one. Figure 4.3 shows this sequence.

```

before-snapshot -> install(top-level) -> after-snapshot
diff = after \ before \ {top-level}
for each dep in diff: SecurityAggregator.analyze(dep)
if any decision == "block":
    for each new dep + top-level: uninstall_package()
    return False (rollback complete)

```

Figure 4.3: Transitive dependency scan and rollback sequence

4.5.3 OSV CVSS Parser

The OSV API returns severity information in two formats. Some records provide a plain numeric CVSS score (`"score": 9.8`). Others provide a full CVSS vector string (`"score": "CVSS:3.1/AV:N/AC:L/..."`). The original implementation attempted `float(score.split('/')[0])`, which silently produced `'unknown'` for every CVSS vector because the last path segment is a metric abbreviation, not a number.

The fix applies a four-step resolution chain:

1. Attempt `float(score)` directly. On `ValueError`, the score is a vector.
2. Fall back to `database_specific.cvss_score` or `severity_score` (numeric).
3. If absent, check `database_specific.severity` (text label such as `"HIGH"`).
4. If still unresolved, iterate `affected[].ecosystem_specific.severity`.

Only if all four steps fail does the severity remain `'unknown'`.

4.5.4 Fail-Closed Exception Handling

`_security_scan()` wraps the entire scan pipeline in a bare `try/except Exception`. If any exception propagates out of `SecurityAggregator.analyze()` or the hash download, the method sets `_last_security_scan` to a synthetic block result and returns `False`. A network timeout, a provider API change, or any other unexpected error blocks the installation rather than allowing it through.

The `-no-security` flag is the only way to bypass this: it skips the scan entirely and passes through to the manager without calling `_security_scan()`. The fail-closed policy is escapable by explicit user choice, but not by accident.

Chapter 5

Testing and Evaluation

5.1 Introduction

This chapter describes the test strategy, the test suite, and the manual evaluation carried out against packages with known security profiles.

5.2 Testing

5.2.1 Test Strategy

The test suite is in `cli-package-manager-unifier/tests/` and uses `pytest`. All tests run without live network connections: HTTP calls are mocked using `unittest.mock.patch` and `monkeypatch`. The shared `conftest.py` fixture monkeypatches `BasePackageManager.is_available` to return `True` for all managers, preventing subprocess calls to actual executables.

Three test levels are used. Unit tests verify individual functions in isolation. Integration tests verify that CLI handler methods invoke the correct manager methods with the correct arguments. Regression tests verify the four architectural security properties against specific test vectors.

5.2.2 Test Suite Summary

Table 5.1: Test Suite Summary by Module

Module	Tests	Focus
test_managers.py	33	BasePackageManager, NPMManager, PipManager
test_cli_handlers.py	23	install/upgrade/uninstall/list handlers
test_cli_integration.py	9	End-to-end CLI arg parsing and dispatch
test_virustotal.py	3	Hash computation, download helpers, API
test_security_aggregator.py	5	Decision scoring, cache key generation
test_security_cache.py	2	TTL expiry, persistence
test_security_providers.py	2	OSV, GitHub Advisory, OSS Index, VirusTotal
test_security_report_and_cache.py	7	JSON and Markdown report writing
test_security_fixes.py	30	TOCTOU, transitive scan, DB, OSV parser, fail-closed
Total	114	

5.2.3 Regression Tests for Security Properties

test_security_fixes.py tests properties that are invisible in normal operation but critical to the security model.

For the TOCTOU fix, the test records the call order of `_resolve_scan_version` and `download_package_and_get_hash` and asserts resolution always precedes download. It also asserts that the version passed to download matches the version embedded in the install specifier. Listing 5.1 shows a representative case.

```

1 def test_version_resolved_before_hash_download(self, cli,
2     monkeypatch):
3     call_order = []
4
5     def fake_resolve(pkg, mgr, *, action="install"):
6         call_order.append("resolve")
7         cli._last_resolved_version = "1.2.3"
8         return "1.2.3"
9
10    def fake_download(pkg, mgr, version=None):
11        call_order.append(f"download:{version}")

```

```
11         return "abc123"
12
13     monkeypatch.setattr(cli, "_resolve_scan_version",
14                           fake_resolve)
15     monkeypatch.setattr("src.cli.download_package_and_get_hash",
16                           fake_download)
17     cli.security_aggregator.analyze = MagicMock(return_value={
18         "decision": "allow", "reason": "ok",
19         "coverage": 2, "counts": {}, "findings": [], "providers"
20         : {}
21     })
22     cli._security_scan("requests", "pip3", "install")
23
24     assert call_order[0] == "resolve"
25     assert "download:1.2.3" in call_order
```

Listing 5.1: TOCTOU test: resolution must precede download

For transitive scanning, the tests mock `list_packages` to return different before and after snapshots and verify the diff is computed correctly. One test injects a blocked transitive dependency and verifies both the transitive package and the top-level package are uninstalled.

For the OSV parser, five test cases cover: CVSS vector with numeric `database_specific` score, CVSS vector with text label, CVSS vector with `ecosystem_specific` fallback, plain numeric string, and plain float. All five assert the output severity is not `'unknown'`.

For fail-closed behaviour, the tests mock `SecurityAggregator.analyze` to raise `RuntimeError` and `ConnectionError`, then assert that `_security_scan` returns `False` and the manager's `install` method is never called.

5.3 Evaluation

Manual evaluation was carried out against packages chosen to exercise the allow and warn decision paths using packages with known security profiles. The block decision path is covered by dedicated regression tests (see Section 5.2.3) rather than live-registry evaluation, because critically-vulnerable packages at affected version numbers change availability over time and cannot be reliably reproduced. Table 5.2 shows the results.

Table 5.2: Manual Evaluation — Known Vulnerable Packages

Package	Manager	Known Status	Decision	Correct
requests	pip3	Clean at 2.31.0	allow	Yes
lodash	npm	CVE-2019-10744 affects <4.17.12; tested at 4.17.21	allow	Yes
django	pip3	Clean at 4.2.13	allow	Yes
pillow	pip3	CVE-2023-44271 affects <10.0.0; tested at 9.5.0	warn (high)	Yes
event-stream	npm	Historic malicious ver.; tested at 3.3.4 (clean)	warn (prov.-dep.)	Partial
minimist	npm	CVE-2021-44906 affects <1.2.6; tested at 1.2.5	warn (high)	Yes

Reproducibility note (NFR3): All rows were evaluated on 21 April 2026 with `VIRUSTOTAL_API_KEY` and `GITHUB_TOKEN` set (free tier); `OSSINDEX_*` credentials were absent. Results are therefore consistent with a two-to-three-provider auth configuration. Outcomes may differ if OSS Index credentials are added or if provider advisory data changes after this date.

The event-stream case deserves explanation. The malicious payload existed only in versions 3.3.6 and 4.0.1, published briefly in 2018 before removal. The version tested — 3.3.4 — is clean. The tool returned warn (provider-dependent), most likely because certain advisory databases retain a package-level advisory that has not been scoped down to the two withdrawn version identifiers. This is consistent with a false positive on a clean version caused by stale advisory retention, not a false negative on an unavailable malicious version; tracing the exact advisory record across all four providers and confirming its precise version scope was outside the scope of this evaluation and is noted as a documented limitation. Providers that do not retain the advisory returned no findings, producing the provider-dependent outcome noted in the table.

The remaining five rows produced correct outcomes. For packages whose tested version was clean, the tool correctly returned allow. For packages whose tested version still contained a known vulnerability, the tool correctly returned warn. The event-stream result reflects an advisory-database scoping limitation that is independent of the scanner’s logic.

Separately, the withdrawn malicious versions (3.3.6, 4.0.1) are no longer served by the npm registry and therefore cannot be tested directly. Any false-negative rate for actively withdrawn malicious artifacts is an inherent limitation of advisory-database scanning and is not captured by the current evaluation table.

A direct comparative measurement against post-install tools such as `npm audit` and `pip-audit` — which would be required to answer RQ3 quantitatively — was not conducted within this evaluation. Designing a controlled experiment that isolates the

timing and detection advantage of pre-install scanning over post-install auditing constitutes a scope limitation and is identified as a direction for future work. All manual evaluations were also conducted with `VIRUSTOTAL_API_KEY` set; the claimed graceful degradation to a three-provider configuration when the key is absent was not exercised during evaluation and constitutes a further untested code path.

Chapter 6

Conclusions

6.1 Introduction

This chapter assesses what the project delivered, where it fell short, and what future work would be most valuable.

6.2 Reflection

The four architectural security properties—TOCTOU fix, transitive scanning with rollback, consistent package database, and fail-closed exception handling—were all implemented and verified by regression tests. The multi-provider approach provides real redundancy. In the reported manual evaluation, OSV.dev and GitHub Advisory covered every observed vulnerable-package case. VirusTotal’s artifact-hash approach is still worth including because it can detect packages where the downloaded file contains malicious content that metadata-only advisories would not flag; however, that capability was not demonstrated by a live-registry case in the current evaluation table and rests on a synthetic test scenario. The JSON TTL cache (O5) was implemented and its TTL expiry and persistence behaviour verified by the `test_security_cache.py` module. Support for the npm, pip3, yarn, and pnpm ecosystems (O6) was delivered in full and exercised throughout the integration tests and the manual evaluation in Section 5.3.

The TOCTOU fix was more complex than expected. Version pinning requires constructing ecosystem-specific specifiers (`==` for pip, `@` for npm), and handling scoped npm packages (`@babel/core`) without mistaking the leading `@` for a version separator. The final regex was the third attempt at a pattern that handled all these cases.

Transitive scanning is slow. Each new dependency introduced by an install triggers a full provider fan-out via the same `SecurityAggregator.analyze()` path used for direct installs. Installing `django` on a fresh environment takes roughly 45 seconds

because of the two transitive dependencies (`sqlparse`, `asgiref`) and the four-provider fan-out each (fewer if optional API keys are absent). The cache helps on repeated operations, but the first install is slow and users notice. A contributing factor is VirusTotal's free-tier rate limit of four lookups per minute (Table 3.1). For packages with more than four transitive dependencies, the retry logic introduces additional waiting time; the recommendation in Section 6.3 to use the free-provider path for transitive scanning would eliminate this constraint entirely.

The OSV CVSS parser bug was not caught until a test was written that injected a real OSV response payload. This confirms the value of using realistic test data. The bug caused every CVSS-vector-format severity to be classified as `'unknown'`, which would have been downgraded to warn rather than block, potentially allowing critical packages through.

6.3 Recommendations

Background transitive scanning with the free-provider path (OSV + GitHub Advisory, no VirusTotal hash) would reduce install latency significantly. The `_scan_free_providers` method already implements this path; it just needs to be wired into the transitive scan loop.

Continuous monitoring of installed packages would catch vulnerabilities published after installation. The `list` command already runs a quick free-provider scan on each package. A daemon mode that ran this daily and emitted a warning report would provide ongoing coverage without requiring a reinstall.

Support for additional ecosystems would extend coverage. The manager abstraction makes adding a new manager a contained task: implement the six abstract methods in `BasePackageManager` and add the class to the `managers` dict in `UnifiedCLI.__init__`. Cargo (Rust) and RubyGems are natural next targets, since OSV.dev already covers both.

A controlled comparative evaluation against post-install tools such as `npm audit` and `pip-audit` would provide the empirical evidence needed to answer RQ3. Such a study would measure detection latency and false-negative rate under a simulated attack scenario in which a malicious package is published between the scan and install steps, comparing both approaches under controlled conditions.

Bibliography

- [1] Cybersecurity and Infrastructure Security Agency, “Alert AA20-352A: Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations,” CISA, Dec. 2020. [Online]. Available: <https://content.govdelivery.com/accounts/USDHSCISA/bulletins/2b1b32d>
- [2] A. Osmani, “I don’t know what to say,” GitHub issue, event-stream repository, Nov. 19, 2018. [Online]. Available: <https://github.com/dominictarr/event-stream/issues/116>
- [3] npm, Inc., “About npm,” *npm Docs*, 2023. [Online]. Available: <https://docs.npmjs.com/about-npm/>
- [4] P. Ladisa, H. Plate, M. Martinez, and O. Barais, “A Taxonomy of Attacks on Open-Source Software Supply Chains,” in *Proc. IEEE Symp. Security and Privacy*, 2023.
- [5] Apache Software Foundation, “Log4j 2 Vulnerability Guidance,” CVE-2021-44228, Dec. 2021. [Online]. Available: <https://logging.apache.org/log4j/2.x/security.html>
- [6] Google Open Source Security Team, “OSV: An open, precise, and distributed approach to vulnerability databases,” 2021. [Online]. Available: <https://osv.dev>
- [7] M. Zimmermann, C. Staicu, C. Tenny, and M. Pradel, “Small World with High Risks: A Study of Security Threats in the npm Ecosystem,” in *Proc. USENIX Security Symp.*, 2019.
- [8] R. Duan, O. Alrawi, R.P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, “Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages,” in *Proc. NDSS*, 2021.
- [9] I. Sommerville, *Software Engineering*, 10th ed. Boston: Addison-Wesley, 2016.
- [10] Sonatype, “OSS Index — Free Open Source Vulnerability Database,” 2024. [Online]. Available: <https://ossindex.sonatype.org>

- [11] VirusTotal, “VirusTotal API v3 Overview,” 2024. [Online]. Available: <https://docs.virustotal.com/reference/overview>
- [12] Python Software Foundation, “PyPI statistics,” 2024. [Online]. Available: <https://pypi.org/stats/>
- [13] Datadog Security Labs, “Compromised axios npm package delivers cross-platform RAT,” Mar. 30, 2026. [Online]. Available: <https://securitylabs.datadoghq.com/articles/axios-npm-supply-chain-compromise/> Accessed: Apr. 23, 2026.
- [14] The Hacker News, “SAP npm Packages Compromised by Mini Supply-Chain Attack,” *The Hacker News*, Apr. 2026. [Online]. Available: <https://thehackernews.com/2026/04/sap-npm-packages-compromised-by-mini.html> Accessed: May 2026.
- [15] L. Abrams, “Bitwarden CLI npm Package Compromised to Steal Developer Credentials,” *BleepingComputer*, Apr. 23, 2026. [Online]. Available: <https://www.bleepingcomputer.com/news/security/bitwarden-cli-npm-package-compromised-to-steal-developer-credentials/> Accessed: May 2026.

Glossary

Artifact hash A SHA-256 digest of the downloaded package file (tarball or wheel), used by VirusTotal to check file reputation.

CVSS Common Vulnerability Scoring System. A numeric scale (0–10) and vector string representation for vulnerability severity.

Fail-closed A safety design where a failure in the security check blocks the operation rather than allowing it through.

GHSA GitHub Security Advisory identifier.

Package manager A tool that automates the download, installation, and removal of software packages (npm, pip3, yarn, pnpm).

PURL Package URL. A standardised string for identifying a software package by ecosystem, name, and version.

Supply-chain attack An attack that targets the tools, libraries, or update mechanisms a development team depends on, rather than targeting the final application directly.

TOCTOU Time-of-Check to Time-of-Use. A race condition where the state of a resource changes between when it is checked and when it is used.

Transitive dependency A package that is not directly requested but is installed because a directly-requested package depends on it.

TTL Time-To-Live. The duration for which a cached entry is considered valid.

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
CISA	Cybersecurity and Infrastructure Security Agency
CLI	Command-Line Interface
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
GHSA	GitHub Security Advisory
OSV	Open Source Vulnerabilities
PURL	Package URL
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TOCTOU	Time-of-Check to Time-of-Use
TTL	Time-To-Live
VT	VirusTotal

Appendix A

CLI Command Reference

A.1 Setup

```
pip install -e .  
# or without installing:  
python unified.py <command>
```

A.2 Commands

`unified list` List all installed packages. Runs a free quick-scan (OSV + GitHub Advisory) and shows a vulnerability label for each package.

`unified install <pkg> -m <manager>` Install a package. Full security pipeline runs first; user is prompted on WARN.

`unified install <pkg> -m <manager> -no-security` Install without any security scan.

`unified install <pkg> -m <manager> -force` Override a BLOCK decision. Runs the full scan but ignores the gate outcome.

`unified upgrade <pkg> -m <manager>` Upgrade to the latest version with full security scan.

`unified upgrade` List all outdated packages across all available managers.

`unified upgrade -all -dry-run` Scan all outdated packages at their target versions without installing. Exit code 1 if any are BLOCK or WARN. Intended for CI gates.

`unified upgrade <pkg> -show-findings 5` Show up to 5 individual vulnerability findings in the scan output.

`unified uninstall <pkg> -m <manager>` Uninstall a package and remove it from PackageCacheDB.

`unified search <query>` Search the npm registry or PyPI for packages matching the query.

A.3 Environment Variable Quick Reference

<code>VIRUSTOTAL_API_KEY=<key></code>	<code># enables VirusTotal hash scanning</code>
<code>OSSINDEX_USERNAME=<user></code>	<code># OSS Index authentication</code>
<code>OSSINDEX_TOKEN=<token></code>	<code># OSS Index authentication</code>
<code>GITHUB_TOKEN=<token></code>	<code># raises GitHub Advisory rate limit</code>
<code>SECURITY_CACHE_TTL_SECONDS=3600</code>	<code># override default 600 s cache TTL</code>